

Where a Unit's State Lives

The Ledger

1 The Question

A unit is anything that can be held: a future, a bond, a mandate, a strategy contract. A wallet holds it. A unit's state — its terms, its lifecycle, a holder's cost — lives somewhere. This fixes where. The placement makes one source of truth, conservation, and deterministic replay attainable. Each economic fact then has exactly one home and one writer, so internal reconciliation failure cannot arise; the sealed single-writer discipline, and the purity and totality of replay, make conservation and deterministic replay hold by construction.

2 The Answer

Every economic relationship a wallet has that reifies as a single unit it holds — a holding, a mandate, a strategy — has its state in three homes, held in two maps. The reification, and the scope it bounds, are shown in §3.

One rule governs every placement: each economic fact has exactly one home and one writer — single source of truth, so internal reconciliation failure cannot arise. A fact fails the rule in only two ways: keyed wrong, one home collapses two facts or one fact duplicates across homes that drift; authored wrong, two authorities keep one fact and their records drift apart. Each of the two questions forestalls one failure, and because there are only these two, there is no third question: whether the fact depends on the holder, and who authors it. §3 works each failure on a concrete unit.

Does the fact depend on the holder, or only on the unit? The scope is one unit's state, so the unit and a holder of it are the only terms in view. By the reification, every economic fact — one that enters conservation, valuation, or profit and loss — is a (holder, unit) fact or a unit fact: a fact about a relationship is a fact about the wallet's position in the unit that relationship is, never about a wider key. What stays keyed by the wallet alone — KYC, permissions, an audit cursor — enters none of the three reckonings; it is identity, not economic state.

Is the fact's record owned by an outside authority or by the ledger? The criterion is who owns the record's history, not who sources the number. An externally authored fact is one an outside authority — the exchange, the contract, the reference-data provider — owns and restates, and the ledger preserves version by version. A ledger-authored fact is one the ledger's own events produce and overwrite.

The two questions give a 2×2 ; three of its four cells are occupied. Each occupied cell is a home: one kind of state.

	ledger-authored	externally authored
per unit	Status: lifecycle stage, last settlement price, current weights, benchmark level	Terms: multiplier, currency, expiry, strike, ISIN, fee schedule, mandate text, benchmark identity
per (holder, unit)	Position: held quantity, high-water mark	— empty —

Each cell names a category; the listings model a slice of it — for Status, the lifecycle stage carrying the settlement price, with current weights and benchmark level elided as the Position row's further fields are. One criterion sorts the whole category: a benchmark level the ledger records and overwrites is ledger-authored, while the benchmark identity the provider restates is the authority's versioned record — the one externally authored, the other not, though both come from that provider.

The fourth cell — an externally authored (holder, unit) fact — is empty (§3).

Terms and status share the unit key but cannot be one value (§3); they ride together as a pair — a third home, not a third map (§4).

3 Why Three

Each occupied cell exists for one concrete reason; the empty cell is empty for one.

Position is keyed by (holder, unit) because two holders of one contract hold different state. A buyer and a seller hold the same future. The buyer's held quantity is +1000; the seller's is -1000, and their costs differ too. A unit-keyed value would store one number and collapse the two. The holder must be in the key.

Status is keyed by the unit alone because one value is read identically by every holder. A future settles at one price on the day; an index stands at one level. Stored per holder, the number is copied across thousands of rows free to drift apart — a reconciliation break by construction. Keyed by the unit, it is one value with one writer.

Terms are a home distinct from status because the two have different authorities of record. Terms are externally authored, status ledger-authored (§2). Co-mingling the authority's record with the ledger's own is the single-source-of-truth violation the system exists to prevent, so the two are distinct values: a non-empty version list grown by `appendVersion`, a single value replaced by `settle`. The amendment event that grows the list is out of scope here, so within this file every terms value has exactly one version, and the distinction is exercised only as the contrast between its two writers.

The fourth cell is empty because no authority issues a fact about one holder's position. A position is the ledger's own from origin: the held quantity folded from the move stream, a high-water mark written by the valuation event, a write-once entry NAV (elided here) folded from the opening event. A custodian holding statement or a prime-broker position report is a reconciliation input the ledger checks against, not a record it adopts as an authority's own. So the (holder, unit) key carries only ledger-authored facts, and the externally authored cell is empty by construction.

A managed account looks like the counterexample: its high-water mark and accrued fee appear to demand a wallet-keyed home. They do not. The manager issues a mandate unit to the client — -1 to the manager, $+1$ to the client, summing to zero like any issued unit — so the client’s state under that mandate is a (client, mandate-unit) position, its high-water mark and entry NAV facts about that position. Two mandates are two rows; a wallet-keyed value would merge them. This discharges the reification for one mandate. A relationship spanning several instruments that does not likewise reduce to a single (holder, unit) row lies outside the scope of §2: its fact would be a (holder, several-units) fact occupying a fourth home and a third map, which the count excludes.

4 The Construction

The homes are built from the smallest pieces, each forced by the one before. The Haskell is `States.hs`; the listings reproduce its declarations, deriving clauses elided.

A balance is a quantity in exact minor units. It has a zero and adds associatively — a monoid — and every balance has a negation, making it a group. The negation later forces the two legs of a transfer to cancel.

```
newtype Qty = Qty Integer
instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid Qty where mempty = Qty 0
negQty :: Qty -> Qty
negQty (Qty n) = Qty (negate n)
```

A balance is held by a wallet, of some unit. Two keys name it; the holder is in the key (§3). The map keyed by this pair is assembled below, once its value has grown past a bare quantity; the `Maybe` its lookup returns carries a real distinction, exhibited on the assembled store.

```
newtype WalletId = WalletId String
newtype UnitId = UnitId String
```

Some facts are one per unit — shared status. Status is one stage of the unit’s lifecycle. `UnitStatus` holds one value per unit, shared — read identically by every holder. Its value changes over time, but `UnitStatus` is not a separate source of truth: the immutable event log is. Every change is caused by a logged event; the stored value is overwritten in place only as a read cache. Replaying the events up to any point rebuilds the exact value that held then, so nothing is lost by overwriting and there is no other way to change the value. A value exists from registration onward. The price a settlement fixes enters only as the `Settled` event in the log — captured there as a logged observation — and is data the stage carries: it rides on the `Active` constructor, so “active with no price” and “registered yet priced” are unspellable — the correlation holds by the type, not by a writer trusted to set two fields in lockstep. The lifecycle carries only the two stages its writers reach: `Registered` at registration — a unit recorded in and known to the ledger before any settlement, which an OTC option, a bespoke mandate, or an internal strategy reaches exactly as an exchange instrument does, not admission to an exchange — and `Active` once the unit settles. A price is a number but not a quantity — prices are never added — so `Price` is a separate newtype with neither identity nor inverse, never summed into a balance.

```
newtype Price = Price Integer
data Lifecycle = Registered | Active Price
newtype UnitStatus = UnitStatus { usLifecycle :: Lifecycle }
defaultStatus :: UnitStatus
defaultStatus = UnitStatus Registered -- the initial stage, no price
```

Terms are the authority’s record, grown only by appending. “One or more versions, append-only” is a non-empty list, so a registered unit always has a current version and “registered but versionless” is not representable. The constructor is not exported, so no importer can lay down a fresh one-version value and discard history; `register`, the one writer that lays down version one, refuses a unit already present (below). No door shortens history: the append-only discipline holds by construction, not by convention.

```
data TermsVersion = TermsVersion String -- multiplier, ISIN, expiry, ...
newtype ProductTerms = ProductTerms (NonEmpty TermsVersion) -- constructor not exported
currentTerms :: ProductTerms -> TermsVersion
currentTerms (ProductTerms vs) = NE.last vs
appendVersion :: TermsVersion -> ProductTerms -> ProductTerms
appendVersion tv (ProductTerms vs) = ProductTerms (vs <> (tv :| []))
```

A position carries more than a balance. Its primary field, the held quantity `psBal`, conserves (§5). Beside it rides a per-(holder, unit) high-water mark: `psHwm` witnesses that the `Position` home can carry a non-conserved fact. Its writer — a valuation event out of scope here — leaves it `mempty` throughout this file; no move writes it as a cancelling leg, so it bears no zero-sum invariant, and it keeps the full `Qty` type rather than a stripped newtype like `Price` because its operation is not settled here — no aggregate over holders is claimed. A first touch starts a position at `zeroP`, any further field elided.

```
data PositionState = PositionState
{ psBal :: Qty -- held quantity: primary, conserved (sum over holders, sec.5)
, psHwm :: Qty } -- high-water mark: not conserved; out-of-scope writer, mempty here
zeroP :: PositionState
zeroP = PositionState mempty mempty
```

The three homes, two maps. Terms and status share the unit key, so a unit’s entry is the pair (terms, status). Co-presence is then the shape of the map, not an invariant to police — no “in terms but not status” value can be written, since one entry carries both halves or neither. The position map is the balance map enriched, its value carrying the held quantity in `psBal`. The constructor and the field selectors `ledgerUnit` and `ledgerPS` are not exported, so a `Ledger` is built only by the writers below and read only through the accessors: were a selector exported, record update through it (`1 { ledgerPS = ... }`) would install a non-conserving map without naming the constructor, bypassing the single-writer

discipline. Coherence is carried by the pair, not the seal; the seal is left to keep conservation true by construction. `emptyLedger` has both maps empty.

```
data Ledger = Ledger -- constructor + field selectors not exported
{ ledgerUnit :: Map UnitId (ProductTerms, UnitStatus) -- terms + status, co-present
, ledgerPS   :: Map (WalletId, UnitId) PositionState } -- per (holder, unit); psBal
emptyLedger :: Ledger
emptyLedger = Ledger Map.empty Map.empty
```

Registration and settlement. Registration introduces a unit, writing the pair of its first terms version and default status as one entry; it refuses a unit already present, returning `Nothing`. Settlement adjusts only the status half — `Map.adjust` over the pair’s `snd` — overwriting it with the `Active` stage carrying the day’s price, leaving terms untouched. `appendVersion`, the remaining terms writer, is out of scope (§3), so `register` and `settle` are the only writers exercised; neither touches `psBal`.

```
register :: UnitId -> TermsVersion -> Ledger -> Maybe Ledger
register u tv l
| Map.member u (ledgerUnit l) = Nothing
| otherwise = Just (l { ledgerUnit =
    Map.insert u (ProductTerms (tv :| []), defaultStatus) (ledgerUnit l) })
settle :: UnitId -> Price -> Ledger -> Maybe Ledger
settle u px l
| Map.member u (ledgerUnit l) =
    Just (l { ledgerUnit =
        Map.adjust (\(t, _) -> (t, UnitStatus (Active px))) u (ledgerUnit l) })
| otherwise = Nothing
```

A move on the ledger is a transfer, lifted. A move’s effect is its per-wallet net delta, written into `psBal` and gated on registration. The two legs — `negQty q` at the source, `q` at the destination, exact inverses in `Qty` (above) — are summed per wallet: two distinct wallets net $-q$ and $+q$; a self-move’s legs land on one wallet and cancel to `mempty`. Either way the move sums to `mempty` over wallets.

```
data Move = Move UnitId WalletId WalletId Qty
applyMove :: Move -> Ledger -> Maybe Ledger
applyMove (Move u from to q) l
| Map.member u (ledgerUnit l) =
    Just (l { ledgerPS = Map.foldrWithKey writeNet (ledgerPS l) (netDeltas from to q) })
| otherwise = Nothing
where
    netDeltas f t qty = Map.insertWith (<>) t qty (Map.insertWith (<>) f (negQty qty) Map.empty)
    writeNet w d ps
    | d == mempty = ps -- a zero net writes no row
    | otherwise =
        let cur = Map.findWithDefault zeroP (w, u) ps
        in Map.insert (w, u) (cur { psBal = psBal cur <> d }) ps
```

The `Maybe` guards input, not the balance: `applyMove` returns `Nothing` only for a move naming an unregistered unit, never for a balance that failed to hold — a well-formed move’s two legs cancel by construction, so `Nothing` answers “is this unit known?”, never “did the balance hold?”. A zero-quantity move and a self-move (`from = to`) are both well-formed and accepted, but each nets to `mempty` on every wallet (above), so neither writes a row: *held* means named in a move that nets nonzero on it.

Reading a position. `position` looks up the pair key on `ledgerPS`, the store the system keeps. Its `Maybe` carries the distinction. `Nothing` is *never held* — no key. `Just` a row is *held*, its `psBal` the held quantity; a closed-out holding is `Just` a row with `psBal` zero — *held, and now flat* — because `applyMove`’s first touch writes a row and a close-out leaves it at zero, never deleted. Settlement entitlement and wash-sale lookback answer *never held* and *held and flat* differently; the retained row keeps them apart.

```
position :: Ledger -> WalletId -> UnitId -> Maybe PositionState
position l w u = Map.lookup (w, u) (ledgerPS l)
```

5 Why It Is Right

Conservation. Conservation is an invariant of the writer, not the store type, which can hold a non-conserving assignment. `applyMove` is the only function that touches `psBal`, and it writes the two legs together from one quantity, so each move changes the holding sum by `negQty q <> q = mempty`. `register` and `settle` touch only `ledgerUnit`, never `psBal`, leaving every holding sum unchanged. From `emptyLedger`, whose sum is zero, every event preserves it, so every reachable ledger conserves — the sealed constructor and withheld field selectors leave no other door.

```
netBal :: Ledger -> UnitId -> Qty
netBal l u = foldMap psBal [ p | ((_, u'), p) <- Map.toList (ledgerPS l), u' == u ]
```

Deterministic replay. `apply` dispatches over the three events; `replay` folds.

```
data Event = Registered UnitId TermsVersion | Moved Move | Settled UnitId Price
apply :: Event -> Ledger -> Maybe Ledger
apply (Registered u tv) = register u tv
apply (Moved m)         = applyMove m
apply (Settled u px)     = settle u px
replay :: [Event] -> Ledger -> Maybe Ledger
replay events l0 = foldM (flip apply) l0 events
```

Replay is deterministic because `apply` is a pure, total function of the event and the prior ledger: the same events give the same ledger. `replay` returns `Just` the rebuilt ledger on a well-formed stream and `Nothing` on an ill-formed one — a repeated registration, or a move or settlement on an unregistered unit — the `foldM` halting at the first refusal. Checkpointing is sound by the monadic left-fold law — `foldM` over a concatenation splits at any cut. Registration, settlement, and moves are all events, so replay from `emptyLedger` rebuilds every unit’s terms (one version each, §3), status, and positions, so every view is a projection of the stream. In particular the stored `UnitStatus` is a cache the log can always rebuild:

replaying the events up to any time reconstructs the exact status that held then — total on well-formed prefixes, deterministic, and reconstructible — so overwriting the cell loses nothing the log does not retain, and a logged event is the only thing that can change it. Row retention serves audit, not determinism.